λ_S^3 — Recursion:

Extending λ_S^2 with Recursive Session Types and Loops

Alexandre Zua Caldeira
Independent Researcher
azuacaldeira@gmail.com

March 2026

Abstract

λ_S^{0-2} restrict session types to finite trees. λ_S^3 lifts this restriction: session types may use recursion ($\mu X.\, S \;/\; X$), creating cyclic state spaces. The term language gains a **while** expression for looping over recursive protocols. We prove preservation and progress (but not termination—λ_S^3 programs may diverge). The key formal challenge is that the state space is no longer a tree but a directed graph with strongly connected components (SCCs); the lattice check quotients by SCCs as established in our prior work [1]. We show that λ_S^3 corresponds to BICA Reborn's handling of recursive session types via the placeholder-merge construction in `StateSpaceBuilder`.

1 Introduction

The Iterator protocol demands recursion:

$$S_{\text{Iter}} = \mu X.\, \&\{\mathit{hasNext} : \oplus\{\mathit{TRUE} : \&\{\mathit{next} : X\},\ \mathit{FALSE} : \mathbf{end}\}\}$$

After *next*(), the protocol returns to the initial state (the X): the client calls *hasNext*() again. This cycle cannot be expressed in λ_S^{0-2}, which require finite tree state spaces.

λ_S^3 adds:

1. **Recursive session types**: $\mu X.\, S$ and variable X.
2. **Cyclic state spaces**: the transition function may revisit states.
3. **While expression**: **while** x **do** e loops while session variable x is not at $\bot$.
4. **Non-termination**: λ_S^3 programs may diverge (an infinite iterator never reaches *FALSE*).

What changes.

- Session types: add $\mu X.\, S$ and X to the grammar.
- State spaces: cycles from recursion; lattice check uses SCC quotient [1].
- Typing: add T-While rule.
- Semantics: add R-While-Step and R-While-Done.
- Soundness: preservation and progress hold; termination does not.

Structure. Section 2 recalls λ_S^2. Section 3 presents the syntax extensions. Section 4 gives the type system (10 rules). Section 5 presents the operational semantics. Section 6 proves soundness. Section 7 works through the full Iterator. Section 8 discusses the result.

2 λ_S^2 Recap

λ_S^2 adds selection types, enum returns, and match expressions to λ_S^1. Session types use Branch, Selection, and End (but no recursion). State spaces are finite trees. The key rule T-Match checks each branch under a different selection successor state.

3 λ_S^3 Syntax

3.1 Recursive Session Types

$$S \ ::= \ \&\{m_1 : S_1, \ldots, m_n : S_n\} \ | \ \oplus\{l_1 : S_1, \ldots, l_n : S_n\} \ | \ \mu X.\, S \ | \ X \ | \ \textbf{end}$$

The recursion variable X refers back to the enclosing $\mu X.\, S$. Well-formedness requires that X is *guarded*: it appears only under at least one Branch or Selection constructor.

3.2 Cyclic State Spaces

For recursive types, the state-space builder uses the *placeholder-merge* construction [2]:

1. Allocate a placeholder state s_X for variable X.
2. Build the body S, which may reference s_X.
3. Merge: replace transitions targeting s_X with the entry state of the body, creating a cycle.

The resulting state space is a directed graph (not a tree). The reachability relation defines a partial order on the SCC quotient, which forms a lattice [1].

3.3 While Expression

$$e \ ::= \ \ldots \ | \ \textbf{while}\ x\ \textbf{do}\ e_{body}$$

while x **do** e_{body} repeatedly evaluates e_{body} while session variable x is not at $\bot$. After each iteration, x may be at a different state (but in the same SCC). When x reaches $\bot$, the loop terminates with **skip**.

Remark 3.1. The while construct is the term-level counterpart of $\mu X.\, S$: the recursion in the type corresponds to a loop in the program. In Java, this is a `while` loop around the protocol:

```
while (true) {
    HasNextResult r = iter.hasNext();
    switch (r) {
        case TRUE  -> iter.next();
        case FALSE -> { return; }  // exits loop
    }
}
```

4 Type System

All rules from λ_S^2 carry over. We add two new rules.

$$\textsc{T-While}\ (\star)\quad \frac{\Gamma(x) = C \mathbin{@} s \qquad s \neq \bot \qquad \mathcal{CT};\ \Gamma \vdash e_{body} : T_b \dashv \Gamma' \qquad \Gamma'(x) = C \mathbin{@} s' \qquad s' \in \mathrm{SCC}(s) \cup \{\bot\}}{\mathcal{CT};\ \Gamma \vdash \textbf{while}\ x\ \textbf{do}\ e_{body} : \textbf{Unit} \dashv \Gamma'[x \mapsto C \mathbin{@} \bot]}$$

T-WHILE: the body is checked once. After the body, x must be either at a state in the same SCC as s (loop continues) or at $\bot$ (loop exits). The output environment records x at $\bot$ because, when the loop terminates, x must be at $\bot$.

$$\text{T-WhileSkip } (\star) \quad \frac{\Gamma(x) = C \mathbin{@} \bot}{\mathcal{CT};\ \Gamma \vdash \textbf{while } x \textbf{ do } e : \textbf{Unit} \dashv \Gamma}$$

T-WHILESKIP: if x is already at $\bot$, the while loop body is never entered and the expression evaluates to **skip**.

5 Operational Semantics

$$\text{R-While-Step } (\star) \quad \frac{\sigma(\ell) = s \qquad s \neq \bot}{\langle H,\ \sigma,\ \textbf{while } \ell \textbf{ do } e\rangle \longrightarrow \langle H,\ \sigma,\ \textbf{let } _ = e \textbf{ in while } \ell \textbf{ do } e\rangle}$$

$$\text{R-While-Done } (\star) \quad \frac{\sigma(\ell) = \bot}{\langle H,\ \sigma,\ \textbf{while } \ell \textbf{ do } e\rangle \longrightarrow \langle H,\ \sigma,\ \textbf{skip}\rangle}$$

R-WHILE-STEP: if ℓ is not at $\bot$, unroll one iteration. R-WHILE-DONE: if ℓ is at $\bot$, exit the loop. All other rules are unchanged from λ_S^2.

Remark 5.1 (Non-termination)**.** If the body of the while loop always leaves ℓ in the same SCC (never reaching $\bot$), the loop diverges. This is expected: an infinite iterator never returns *FALSE*. Termination is not a property of λ_S^3.

6 Soundness

Theorem 6.1 (Conservative Extension)**.** *λ_S^3 conservatively extends λ_S^2: when no recursive types or while expressions appear, all λ_S^2 derivations hold.*

Theorem 6.2 (Preservation)**.** *Well-typed configurations reduce to well-typed configurations.*

Proof. New cases:

- R-WHILE-STEP: unfolds to **let** $_ = e_{body}$ **in while** ℓ **do** e_{body}. The body e_{body} is typeable by the premise of T-WHILE; the continuation **while** ℓ **do** e_{body} is typeable because the body leaves ℓ in the same SCC or at $\bot$.
- R-WHILE-DONE: ℓ at $\bot$, result is **skip** : **Unit**. □

Theorem 6.3 (Progress)**.** *Well-typed non-values can always reduce.*

Proof. For **while** ℓ **do** e: either $\sigma(\ell) = \bot$ (R-WHILE-DONE) or $\sigma(\ell) \neq \bot$ (R-WHILE-STEP). Both always apply. □

Corollary 6.4 (Session Completion (conditional))**.** *If a λ_S^3 program terminates (reduces to a value), then all locations are at $\bot$.*

Proof. By T-WHILE, the output environment records x at $\bot$. If the program terminates, all while loops have exited, so all session variables are at $\bot$ by iterated preservation. □

Remark 6.5. Termination is *not* guaranteed. Session completion is conditional on termination. This is a fundamental difference from λ_S^{0-2}, where termination is guaranteed by the finite-tree structure of state spaces.

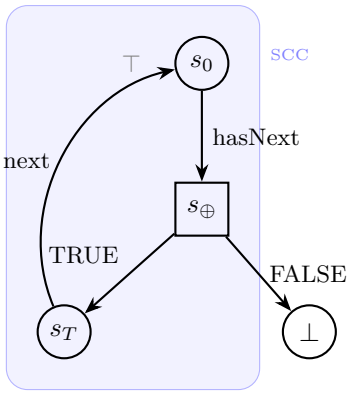

Figure 1: Full Iterator state space. The cycle $s_0 \to s_\oplus \to s_T \to s_0$ forms an SCC. The lattice quotients this SCC to a single node above $\bot$.

7 Worked Examples

7.1 Full Iterator

Session type:

$$S_{\text{Iter}} = \mu X.\,\&\{\mathit{hasNext} : \oplus\{\mathit{TRUE} : \&\{\mathit{next} : X\},\ \mathit{FALSE} : \mathbf{end}\}\}$$

Client program:

$$\begin{aligned} e = {} & \mathbf{let}\ \mathit{it} = \mathbf{new}\ \text{Iter}()\ \mathbf{in} \\ & \quad \mathbf{while}\ \mathit{it}\ \mathbf{do} \\ & \quad\quad \mathbf{let}\ r = \mathit{it}.\mathit{hasNext}()\ \mathbf{in} \\ & \quad\quad \mathbf{match}\ r\ \mathbf{with}\ \{ \\ & \quad\quad\quad \mathit{TRUE} \Rightarrow \mathit{it}.\mathit{next}(), \\ & \quad\quad\quad \mathit{FALSE} \Rightarrow \mathbf{skip}\ \} \end{aligned}$$

Typing.

1. T-New: it at $s_0 = \top$.
2. T-While: body is checked once from s_0.
   (a) T-Call: $\delta(s_0, \mathit{hasNext}) = s_\oplus$.
   (b) T-Match at $s_\oplus$:
      - TRUE: it at s_T. $\delta(s_T, \mathit{next}) = s_0$. After body: it at $s_0 \in \text{SCC}(s_0)$. (*loop continues*)
      - FALSE: it at $s_F = \bot$. (*loop exits*)
3. After while: it at $\bot$. Session complete.

8 Discussion and Conclusion

8.1 Implementation Correspondence

λ_S^3 completes the core of the λ_S hierarchy for single-object, single-threaded protocols. With Branch, Selection, and Recursion, the calculus covers the full class of session types used in practice (minus parallel composition, which is λ_S^4).

Table 1: λ_S^3 correspondence with BICA Reborn.

Concept	**Formal**	**BICA**
Recursive type	$\mu X.\, S$, X	`Rec`, `Var` AST nodes
Cyclic state space	Placeholder-merge	`StateSpaceBuilder.buildRec()`
SCC quotient	SCC(s)	`LatticeChecker` (Tarjan's)
While loop	T-While	Java `while/for`
Non-termination	Possible divergence	Infinite loops in client code

8.2 The λ_S Hierarchy

Calculus	**Session types**	**Term language**	**Terminates?**
λ_S^0	$\&$, end	let, call	Yes (finite trees)
λ_S^1	$\&$, end	+constants	Yes
λ_S^2	$\&$, $\oplus$, end	+match	Yes
λ_S^3	$\&$, $\oplus$, μ, end	+while	**No**

Preview: λ_S^4 (parallel). The final single-object extension adds the parallel constructor $S_1 \| S_2$, enabling concurrent access to shared session-typed objects. This forces the product lattice structure that is the central result of our research programme [1].

References

[1] Alexandre Zua Caldeira. Session types as algebraic reticulates: The lattice theorem (step 5), 2026. Submitted to ICE 2026.

[2] Alexandre Zua Caldeira. State-space construction for binary session types (step 1), 2026. arXiv preprint.